Ilyes khelfi
Evan Zougs

Compte-rendu du projet M@Banque

Sommaire

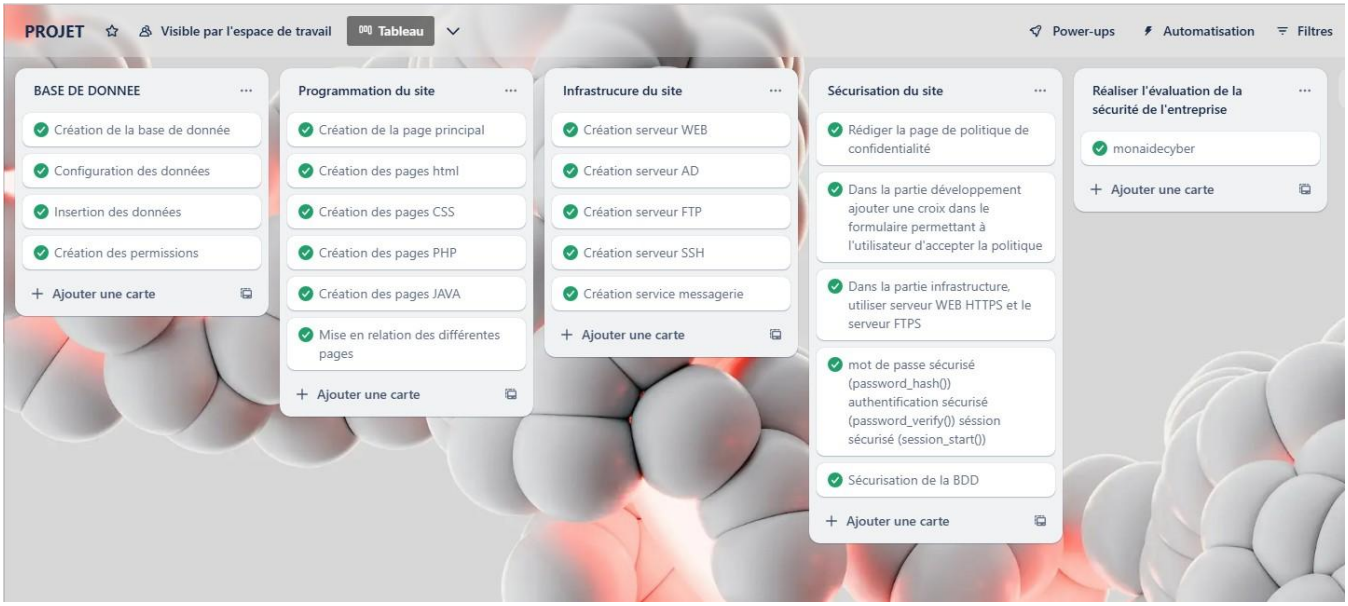
1. Introduction 2
2. Organisation du groupe 2
3. Arborescence du site 2
4.1 Mise en place de l'accès SSH 4
4.2 Installation de la pile LAMP et implémentation de la base de données 5
4.3 Configuration du serveur web en HTTPS 6
4.4 Mise en place du serveur FTPS 7
5. Langages utilisés pour le développement de l'application web du site 8
6. Difficultés rencontrées et solutions apportées 8
7. Mise en place d'un plan de sauvegarde automatisé 8
8. Démarche de cybersécurité du site 9
10. Solution complémentaire mis en place 10

1. Introduction

Dans le cadre de notre projet pédagogique intitulé **M@Banque**, nous avons eu pour mission, en binôme, de simuler l'infrastructure informatique d'un établissement bancaire virtuel. L'objectif principal était de mettre en œuvre une architecture réseau complète intégrant des services essentiels à toute entreprise moderne : hébergement web, messagerie interne, gestion d'annuaire, transfert de fichiers sécurisé, administration distante et sauvegarde automatisée.

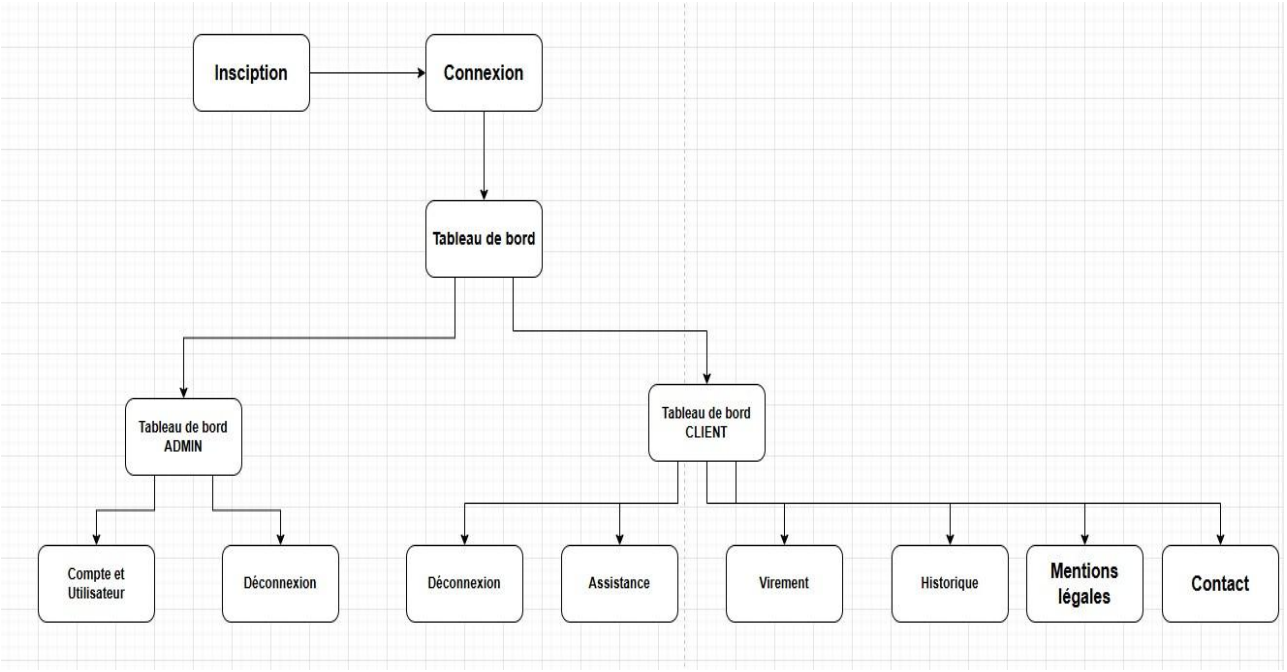
Ce compte-rendu détaille les différentes étapes techniques de ce déploiement, les difficultés rencontrées, les solutions apportées et les enseignements tirés tout au long du projet.

2. Organisation du groupe



Afin de faciliter la gestion de notre projet de groupe, nous avons utilisé **Trello**, un outil de gestion de tâches en ligne. Il nous a permis de répartir clairement les rôles et les responsabilités entre les membres de l'équipe, d'organiser les différentes étapes du projet sous forme de cartes, et de suivre l'avancement des tâches en temps réel. Grâce à ses fonctionnalités de collaboration, nous avons pu rester synchronisés tout au long du développement, améliorer notre organisation et respecter les délais fixés.

3. Arborescence du site



L'arborescence du site *M@Banque*, conçue à l'aide de **draw.io**, a été pensée pour offrir une navigation simple et intuitive aux utilisateurs tout en séparant clairement les rôles (administrateur et client). Elle repose sur un **accès progressif**, en fonction du niveau d'autorisation de l'utilisateur.

Dès l'arrivée sur le site, l'utilisateur est dirigé vers la **page d'inscription**. Cette page permet à un nouveau client de créer un compte en renseignant ses informations personnelles (nom, prénom, adresse e-mail, mot de passe, etc.). Les données saisies sont vérifiées et sécurisées, notamment grâce au hashage des mots de passe côté serveur.

Une fois l'inscription effectuée avec succès, l'utilisateur peut accéder à la **page de connexion**. Celle-ci permet aux clients ou à l'admin déjà enregistrés de s'identifier à l'aide de leur adresse e-mail et de leur mot de passe. En cas d'identifiants valides, l'utilisateur est redirigé vers son espace personnel, dont le contenu dépend de son rôle dans le système.

Si l'utilisateur connecté possède le rôle admin, il accède à un tableau de bord administratif simplifié mais essentiel. Ce tableau de bord donne accès aux fonctionnalités suivantes :

- **Comptes et utilisateurs**
- **Déconnexion**

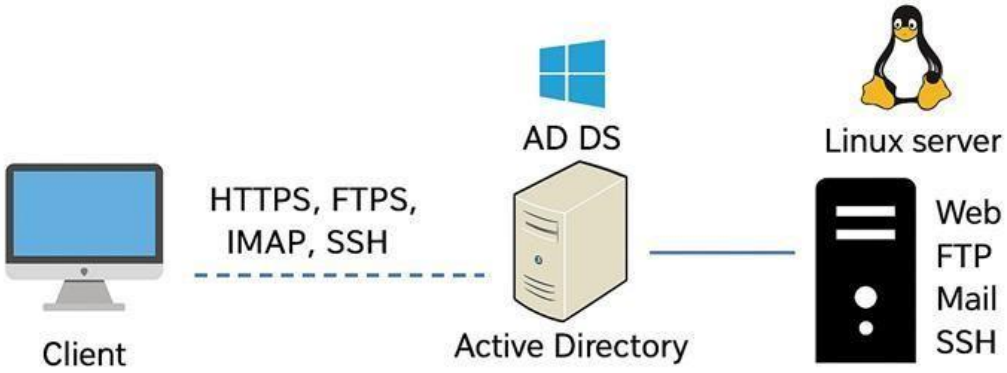
L'interface est volontairement épurée, centrée sur la gestion des utilisateurs et des comptes, dans un souci de clarté et d'efficacité.

Si l'utilisateur est un client, il est redirigé vers un tableau de bord spécifique contenant plusieurs pages utiles à la gestion de son espace bancaire personnel :

- **Virement**
- **Historique**

- **Assistance**
- **Contact**
- **Mentions légales**
- **Déconnexion**

4. Déploiement de l'infrastructure serveur



4.1 Mise en place de l'accès SSH

La première étape a consisté à activer un **accès SSH sécurisé** afin de pouvoir administrer notre serveur Linux à distance. J'ai généré une paire de clés SSH depuis la machine cliente, avec l'objectif d'établir une connexion sans mot de passe, plus fiable et sécurisée.

Cependant, j'ai rapidement été confronté à une erreur de connexion persistante. Après avoir vérifié la configuration du service, il est apparu que le problème venait de la couche réseau : le client et le serveur ne se trouvaient pas sur le même VLAN, ce qui empêchait toute communication. Une fois les interfaces correctement alignées, la connexion a pu s'établir.

Cette situation m’a permis de renforcer ma capacité à analyser les erreurs avec méthode, en commençant toujours par les éléments les plus bas niveau (le réseau), avant de remonter aux couches applicatives.

Pour renforcer la sécurité, j’ai également modifié le port par défaut de SSH et restreint l’accès à des utilisateurs bien définis.

4.2 Installation de la pile LAMP et implémentation de la base de données

La deuxième étape a porté sur l’installation de la **pile LAMP** (Linux, Apache, MySQL, PHP) sur une distribution Debian, dans le but d’héberger le site web bancaire. Apache2 a été configuré pour le service HTTP, PHP pour le traitement des pages dynamiques, et MySQL pour la gestion des données utilisateurs.

Une fois l’environnement LAMP installé et opérationnel, nous avons procédé à la **mise en place de la** base de données MySQL. Celle-ci a été pensée pour stocker les différentes informations nécessaires au fonctionnement du site bancaire.

Parmi les tables principales, on retrouve la table utilisateurs, conçue pour stocker les données d'identification et d'accès des clients et de l’admin.

Dans un souci de **sécurité des données**, notamment dans un contexte sensible comme celui d’un site bancaire, les mots de passe ne sont jamais stockés en clair. Ils sont **systématiquement** hashés **à** l’aide de la fonction password_hash**() en** PHP**.**

Cela garantit qu’en cas de fuite de la base de données, les mots de passe restent illisibles et donc inutilisables pour un attaquant.

Par la suite, des relations ont été créées entre les différentes tables à l’aide de **clés étrangères**, permettant ainsi de relier les utilisateurs à d’autres entités comme leurs comptes bancaires ou leurs opérations.

Des insertions de données ont ensuite été insérés manuellement pour effectuer des tests fonctionnels. Ces tests ont permis de valider l’intégrité de la base, le bon fonctionnement des relations entre les tables, ainsi que la sécurité des identifiants.

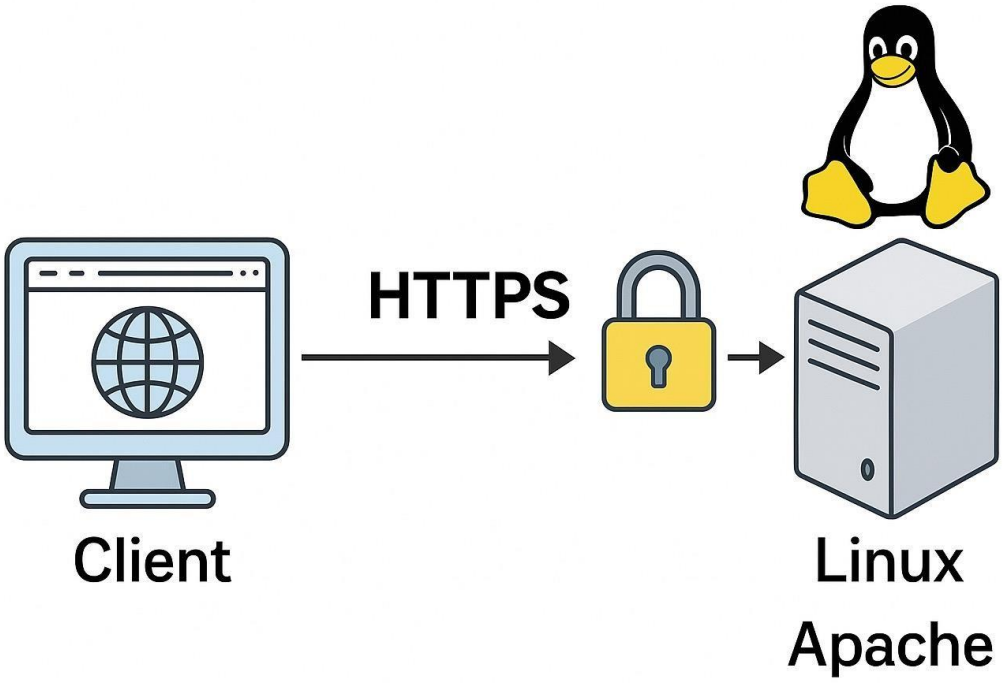
Pour permettre à l'application web bancaire d'interagir avec la base de données, une connexion sécurisée à MySQL via PHP a été mis en place. Cette connexion est essentielle pour effectuer des opérations comme l’inscription, l’authentification des utilisateurs, la consultation des comptes ou encore l’enregistrement de transactions.

La connexion est réalisée à l’aide de l’extension PDO**, q**ui offre une interface plus sécurisée. Elle permet également l’utilisation de requêtes préparées, réduisant considérablement les risques d’injection SQL.

Mais encore, lors de la création de la base, une erreur de syntaxe a été commise, nous avons introduit un caractère interdit dans le nom de la base, ce qui a bloqué l'exécution. Après correction, la base nommée mabanque a été créée sans encombre.

L'importation du fichier .sql transmis par le développeur SLAM a également posé problème, car les droits root n'étaient pas suffisants dans le terminal classique. La solution a été d'ouvrir une session MySQL avec sudo, permettant ainsi l'import correct des données. Cette phase nous a sensibilisé à la gestion fine des droits systèmes et à l'importance de la syntaxe en ligne de commande.

4.3 Configuration du serveur web en HTTPS

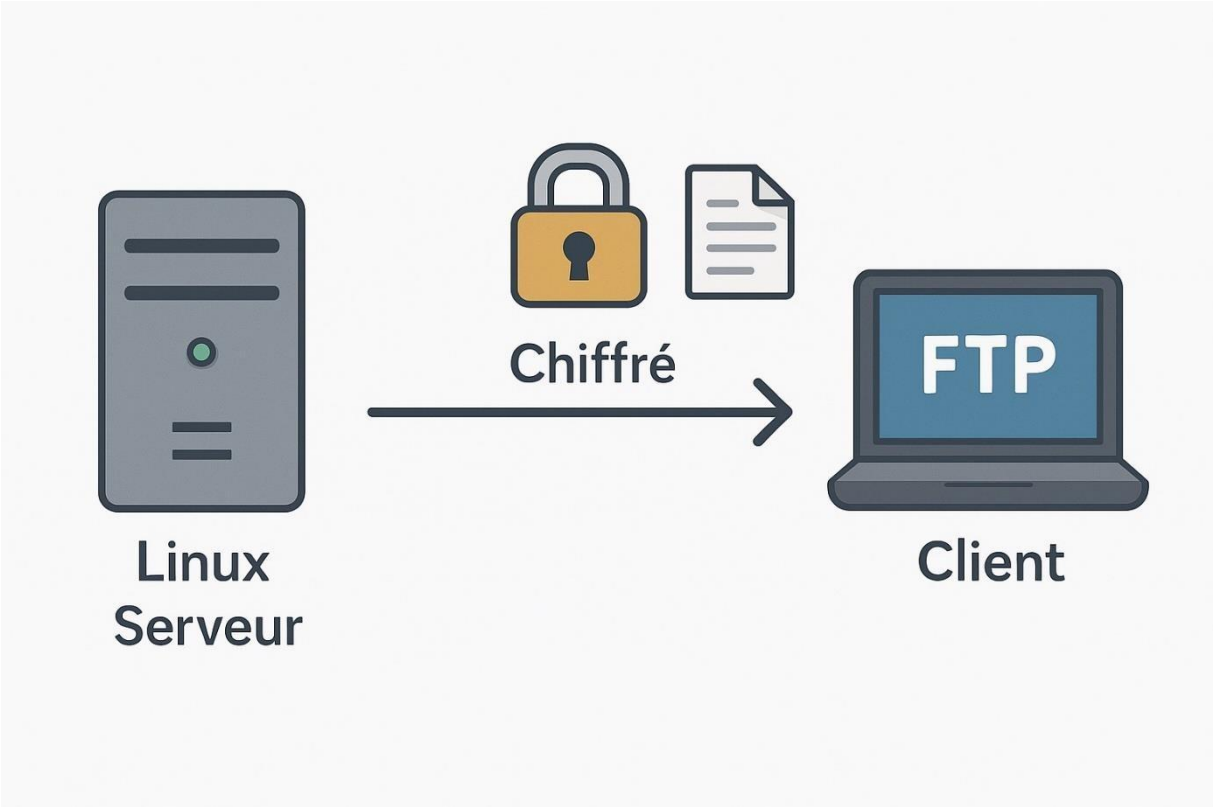


Pour sécuriser les communications entre le client et le serveur web, j'ai mis en place le **protocole HTTPS** sur le serveur Apache. Dans un contexte fermé sans DNS public, j'ai opté pour un **certificat SSL auto-signé** généré via OpenSSL.

Les modules Apache nécessaires (ssl et headers) ont été activés. J'ai ensuite configuré un VirtualHost pour rediriger tout le trafic HTTP vers HTTPS et associer le certificat SSL à l'IP du serveur.

Lors des tests, le navigateur affichait une alerte (certificat non reconnu), mais cela était prévu et n'empêchait pas le chiffrement effectif de la connexion. Cette étape m'a permis de comprendre les bases de la cryptographie SSL/TLS, et l'importance d'offrir un canal de communication sécurisé, même en environnement de test.

4.4 Mise en place du serveur FTPS



Pour permettre le **transfert sécurisé de fichiers** entre le développeur et le serveur, j'ai déployé un serveur FTPS en configurant vsftpd avec TLS. Cela m'a permis de transformer un service historiquement peu sécurisé (FTP) en un canal fiable.

J'ai désactivé l'accès anonyme, généré un certificat dédié au service, et créé un utilisateur restreint à un répertoire spécifique. Le service a ensuite été testé avec FileZilla en mode FTP explicite sécurisé (FTPES), ce qui a confirmé la bonne configuration.

Cette mise en œuvre m'a permis de renforcer ma compréhension des protocoles de transfert, et de l'importance de toujours chiffrer les communications, surtout lorsqu'elles concernent des fichiers critiques.

5. Langages utilisés pour le développement de l'application web du site

Pour le développement du site web bancaire, plusieurs langages complémentaires ont été utilisés afin de construire une application à la fois fonctionnelle, sécurisée et agréable à utiliser. Ces langages sont : HTML, CSS et PHP.

Tout d'abord, le HTML a été utilisé pour structurer les pages du site. C'est le langage de base du web, qui permet de créer les éléments visibles comme les titres, paragraphes ou formulaires. Il sert à organiser le contenu de manière logique afin de garantir une bonne lisibilité à l'utilisateur.

Ensuite, le CSS a été utilisé pour mettre en forme les pages HTML. Grâce au CSS, nous avons pu définir l'apparence visuelle du site : couleurs, polices, marges, boutons, menus, etc. Cela a permis d'obtenir un rendu moderne et clair.

Enfin, le PHP a été utilisé côté serveur pour gérer la logique de l'application. PHP permet d'interagir avec la base de données MySQL, de traiter les formulaires (inscription, connexion, etc.), de sécuriser les accès, et de générer dynamiquement les pages selon les actions des utilisateurs. Grâce à PHP, nous avons pu mettre en place une authentification sécurisée, avec gestion de sessions et hashage des mots de passe.

6. Difficultés rencontrées et solutions apportées

Le projet n'a pas été un long fleuve tranquille. Plusieurs blocages techniques sont survenus, nécessitant analyse, documentation et persévérance :

- **Erreur de syntaxe SQL** : évitée par la suite en respectant strictement les normes de nommage.
- **Échec de l'import SQL** : résolu par un accès administrateur via sudo mysql.
- **Difficulté à affecter les privilèges au développeur sur la base de donnée**
- **Messagerie initialement non locale** : inadaptée à l'environnement, remplacée par un serveur Postfix/Dovecot local.
- **Script cron inactif** : l'exécution échouait à cause d'un oubli du bit exécutable sur le script, corrigé avec chmod +x.

Chaque difficulté a représenté une **opportunité d'apprentissage**, en m'obligeant à approfondir mes recherches et à mieux comprendre le comportement des services.

7. Mise en place d'un plan de sauvegarde automatisé

Dans une logique de continuité d'activité, j'ai mis en œuvre un système de sauvegarde automatique afin de protéger les données critiques de l'infrastructure. J'ai rédigé un script bash nommé backup.sh, placé dans /usr/local/bin/, chargé de sauvegarder chaque jour à 2h du matin :

- le dossier /var/www/html contenant les fichiers du site web,
- les fichiers de configuration Apache et FTP,
- la base de données MySQL nommée mabanque.

Le script archive les fichiers dans un dossier de sauvegarde spécifique, en y ajoutant la date du jour. Il comprend aussi une fonction de **rotation** qui supprime les fichiers de plus de 7 jours afin de ne pas saturer le stockage.

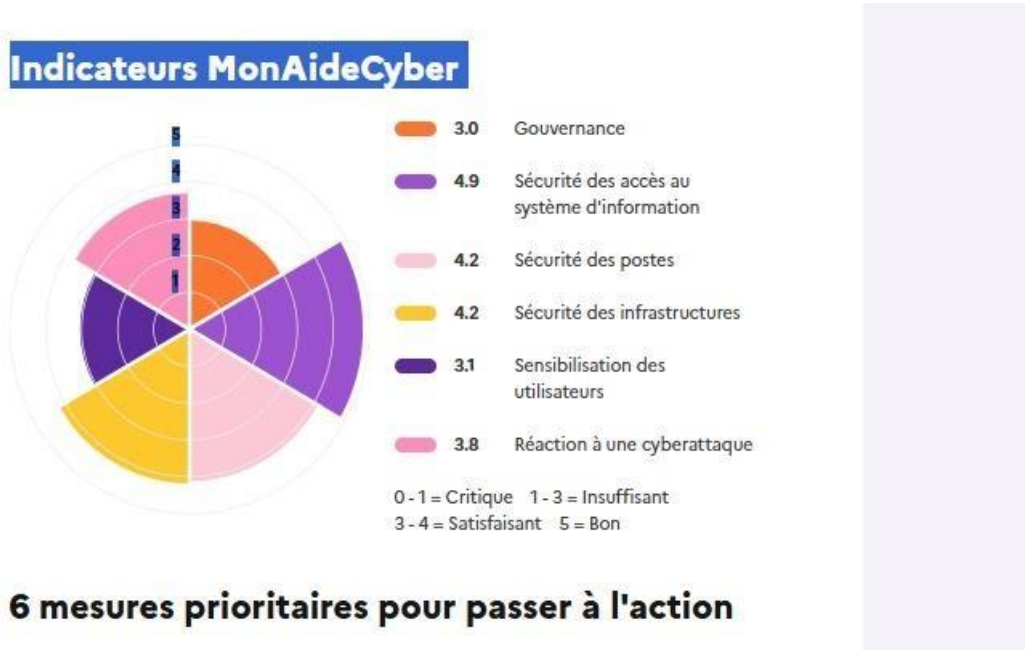
Bénéfices constatés

Ce plan de sauvegarde m'a permis de :

- Mieux comprendre les enjeux de la sécurité opérationnelle,
- Apprendre à automatiser des tâches répétitives avec fiabilité,
- Maîtriser la gestion de la crontab et des droits d'exécution,
- M'assurer qu'en cas de panne, de mauvaise manipulation ou d'attaque, les données puissent être restaurées rapidement.

Cette mesure renforce la **résilience** de l'infrastructure et constitue une **bonne pratique essentielle** dans tout environnement professionnel.

8. Démarche de cybersécurité du site



9. Piste d'amélioration issue de MonAideCyber : Mettre en œuvre une solution d'anti-spam et d'anti-hameçonnage

Dans le cadre de ce projet, une auto-évaluation de la sécurité a été réalisée à l'aide de l'outil **MonAideCyber**, proposé par l'ANSSI. Cet outil a permis d'identifier les points forts de l'infrastructure déployée, mais aussi plusieurs axes d'amélioration. L'un des éléments ressortis comme prioritaire concerne la **protection des messageries contre les spams et les tentatives d'hameçonnage (phishing)**.

Les systèmes de messagerie sont en effet des cibles privilégiées des cyberattaquants. Une simple ouverture de pièce jointe piégée ou un clic sur un lien malveillant peut compromettre l'intégralité du réseau. Aujourd'hui, les attaques sont de plus en plus ciblées et sophistiquées : les mails de phishing ressemblent à s'y méprendre à des messages légitimes, ce qui rend la vigilance humaine parfois insuffisante.

Pour y remédier, il serait pertinent de **compléter notre serveur de messagerie Postfix/Dovecot par une solution anti-spam/anti-phishing**, capable de :

- Filtrer automatiquement les messages malveillants grâce à des bases de signatures ou l'intelligence artificielle ;
- Analyser les liens contenus dans les e-mails ;
- Isoler les messages suspects dans des quarantaines ;
- Fournir des statistiques et alertes de sécurité ;
- Proposer une interface de gestion centralisée des règles de filtrage.

10. Solution complémentaire mis en place

- **Pare-feu UFW**

Même si le pare-feu n'était pas bloquant durant le projet, une configuration de **UFW (Uncomplicated Firewall)** a été envisagée pour restreindre les connexions aux seuls ports nécessaires (22 pour SSH, 443 pour HTTPS, 21/990 pour FTPS, etc.). Cela permet de limiter la surface d'attaque et de mieux contrôler les flux réseau. Par défaut, tout le trafic entrant aurait dû être bloqué sauf les services explicitement autorisés.